

WorkLink

System Design Diagrams & Documentation

On-Demand Skilled Worker & Job Matching Platform

Prepared from the WorkLink Software Requirements Specification (SRS)

Generated: July 27, 2026

Contents

1. Introduction
2. Use Case Diagram
3. Activity Diagram — Job Lifecycle
4. Class Diagram
5. Entity Relationship Diagram (ERD)
6. System Architecture Diagram
7. Recommended Improvements to the Requirements
8. Summary

1. Introduction

This document accompanies the WorkLink Software Requirements Specification. WorkLink connects requesters who need short-term or one-off workers with skilled workers seeking immediate job opportunities, matched by skill, location, and availability. The diagrams below translate the requirements and use cases already documented into the standard design artifacts of Phase 2 (Design) of the project plan: a use case diagram, an activity diagram, a class diagram, an entity relationship diagram, and a system architecture diagram. A set of recommended improvements is included at the end to strengthen the requirements before development begins.

2. Use Case Diagram

The use case diagram below captures every interaction defined for the three actors in the SRS: the Worker, the Requester, and the Administrator. Shared registration and login flows are shown per-actor for clarity, since each actor completes a distinct profile afterward.

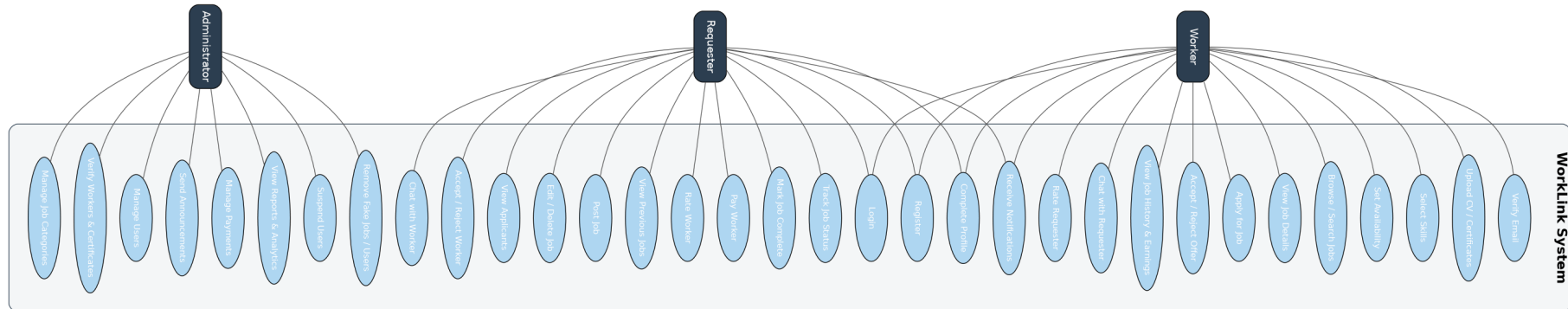


Figure 1 — Use case diagram showing all Worker, Requester, and Administrator interactions with WorkLink.

3. Activity Diagram — Job Lifecycle

This activity diagram models the core end-to-end workflow of the platform: a Requester posts a job, a Worker discovers and applies for it, the Requester reviews the application, and — if accepted — both parties coordinate, the Worker performs the job, and payment and mutual ratings follow completion. A dispute path is included, routing unresolved completions to Administrator review before payment is released, which reflects a recommended improvement described in Section 7.

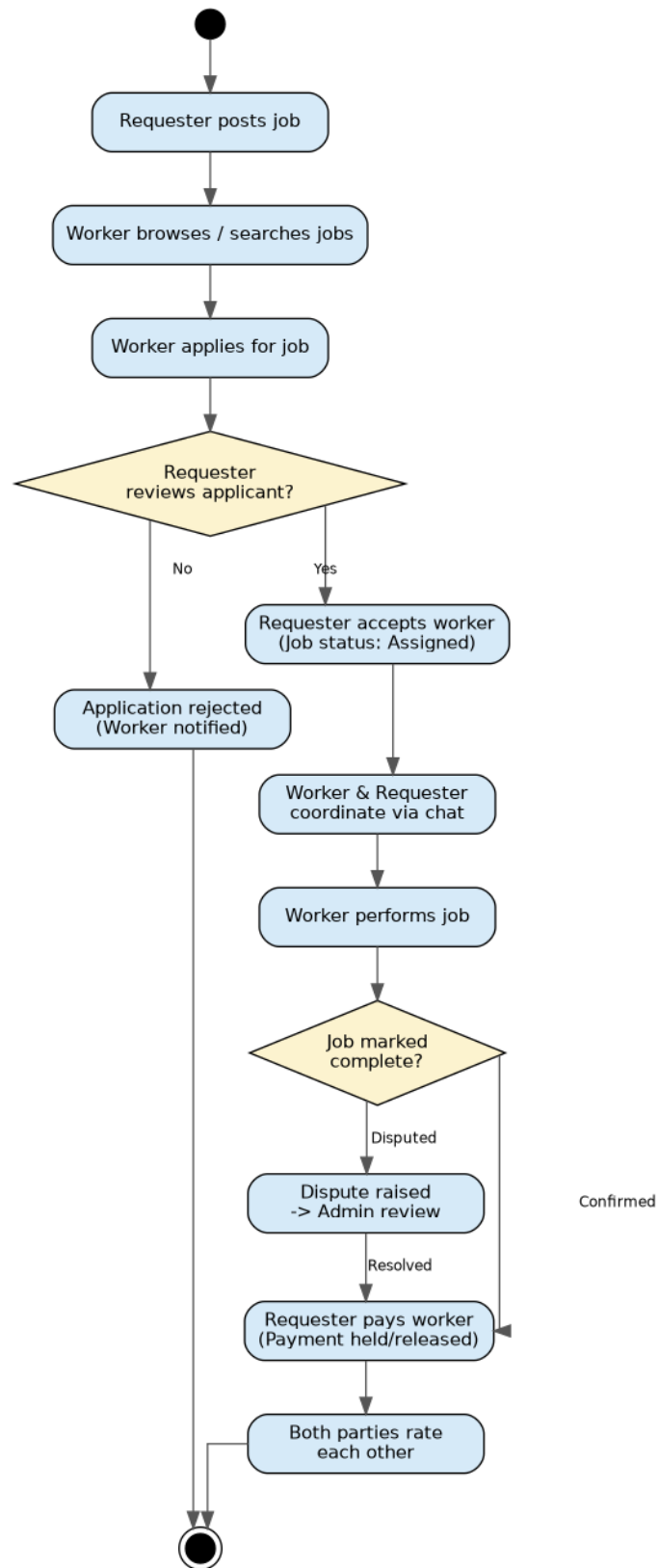


Figure 2 — Activity diagram tracing a job from posting through payment and rating.

4. Class Diagram

The class diagram models WorkLink's core domain objects. Worker, Requester, and Administrator specialize a common User class, reflecting shared authentication and profile behavior while keeping role-specific attributes and methods separate. Job, Application, Payment, Rating, Message, and Notification represent the primary transactional entities described in the functional requirements.

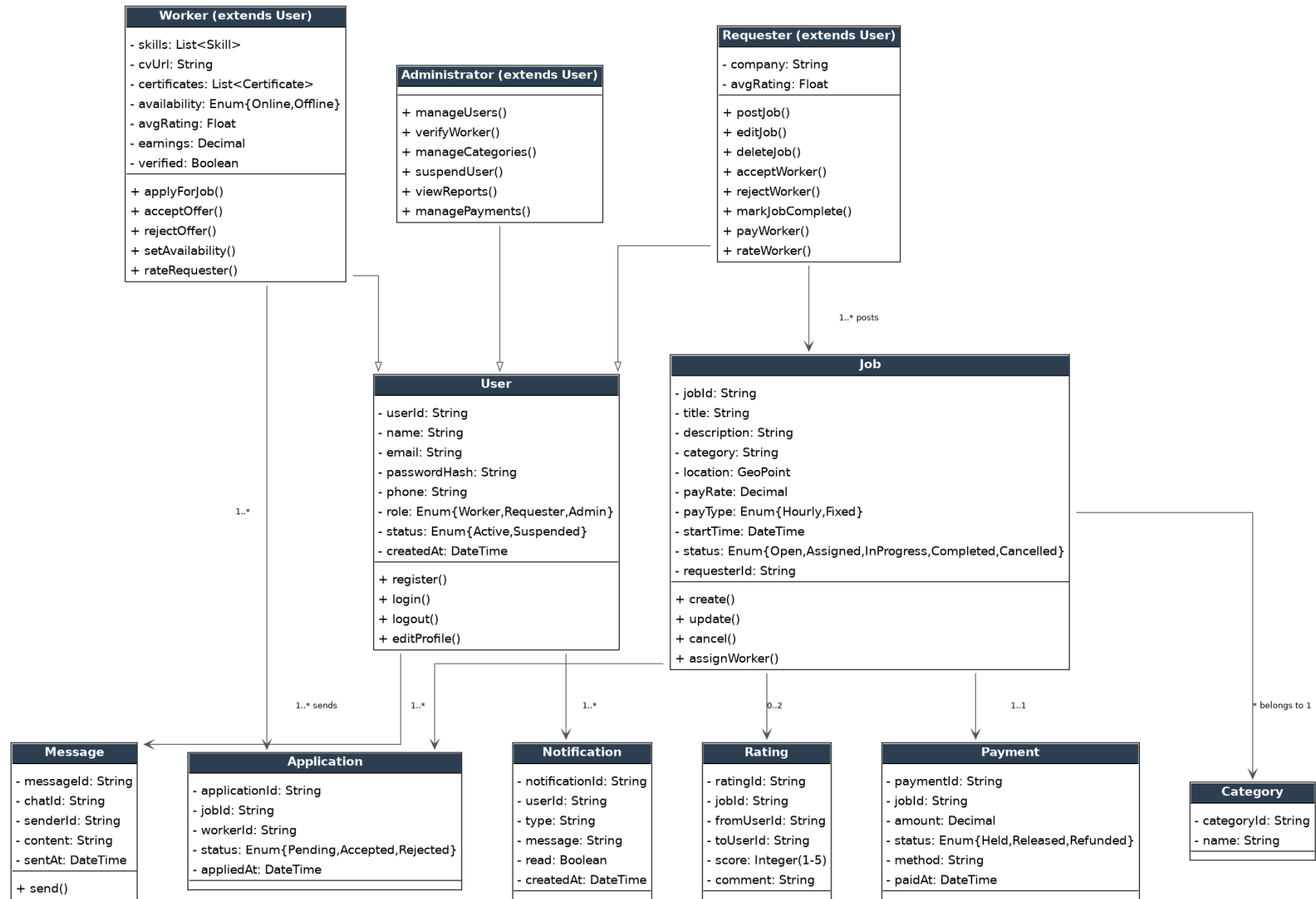


Figure 3 — Class diagram showing core domain entities, attributes, methods, and relationships.

5. Entity Relationship Diagram (ERD)

The ERD translates the class diagram into a relational schema suitable for a document/relational hybrid store such as Firestore, or a traditional relational database if preferred. Primary keys are underlined and bold; foreign keys are italicized. `WORKER_SKILLS` is a many-to-many join table between `WORKER_PROFILES` and `SKILLS`.

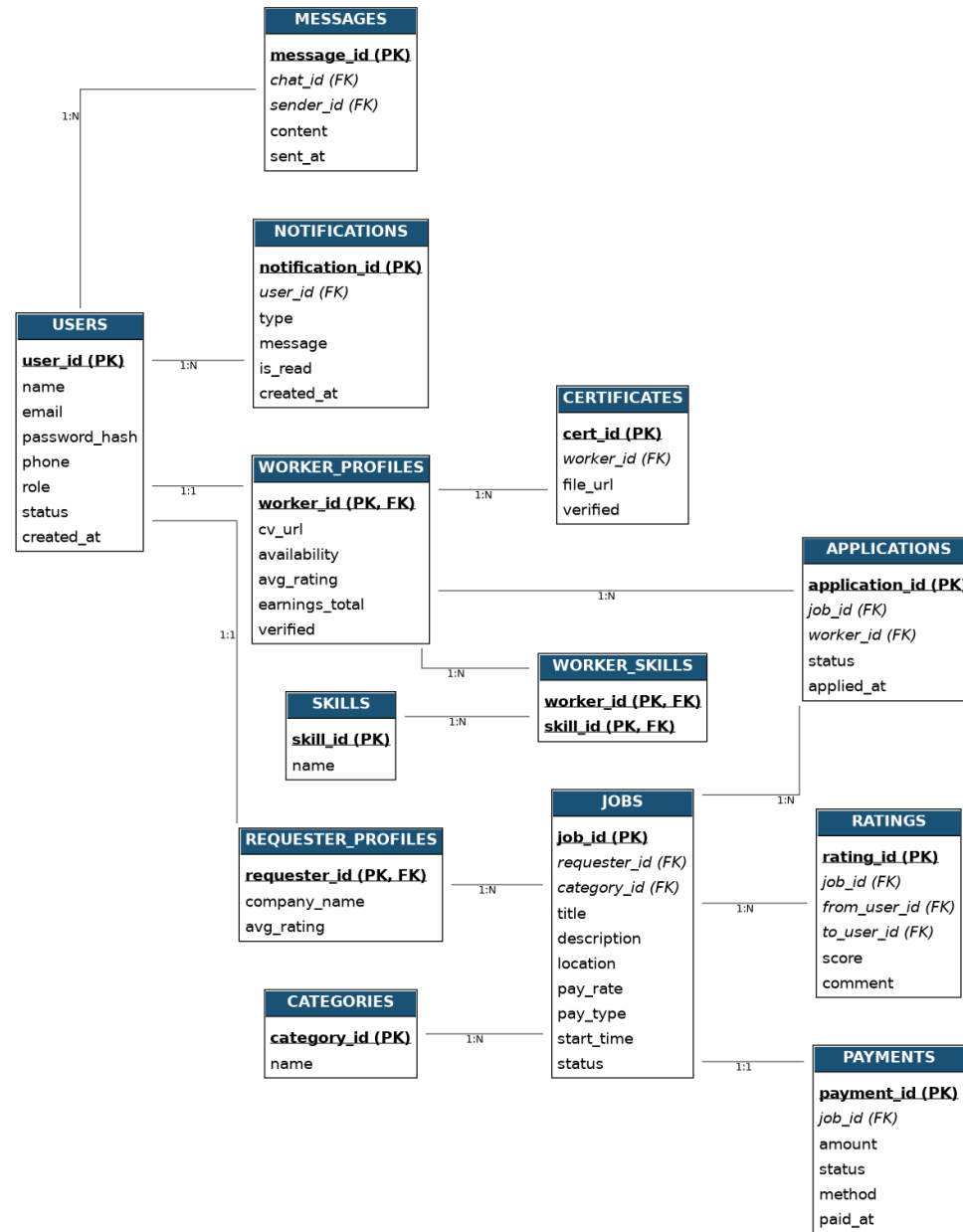


Figure 4 — ERD showing tables, keys, and cardinalities for the WorkLink database.

6. System Architecture Diagram

The architecture follows the technology stack specified in the development plan: a JSP/HTML/CSS/JavaScript client, a Java EE backend exposing REST endpoints and business logic, and Firebase for authentication, data storage, file storage, and push notifications. External services for payments and geolocation are integrated at the business logic layer.

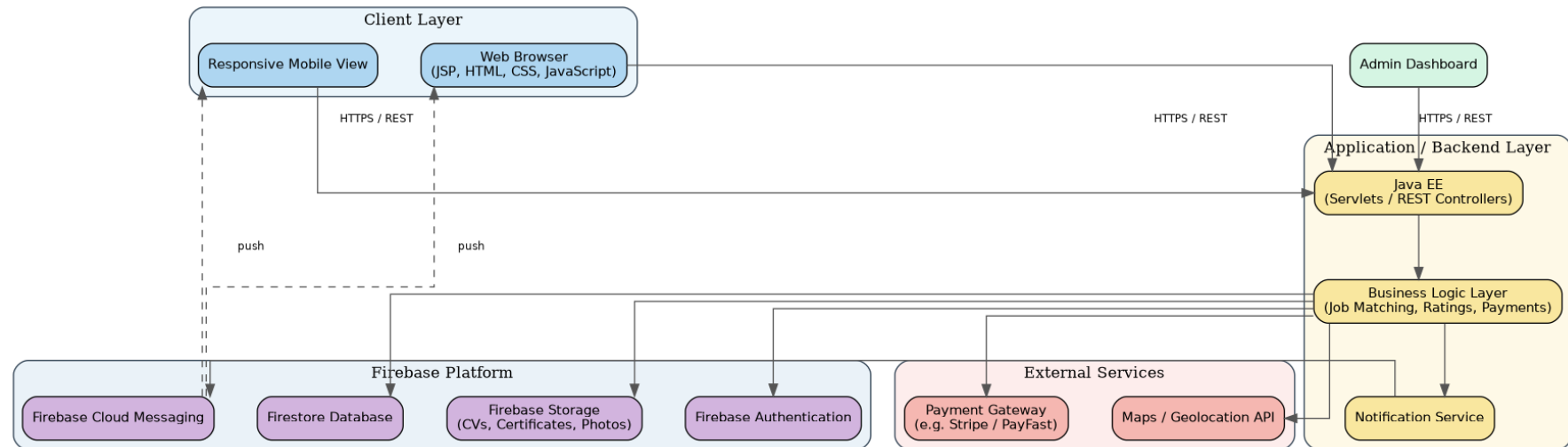


Figure 5 — High-level system architecture based on the Java EE + Firebase technology stack from the SRS.

7. Recommended Improvements to the Requirements

The following suggestions build on the existing SRS without changing its core direction. They are grouped by theme and can be prioritized independently for later phases.

Trust & Safety

- Introduce an escrow-style payment flow: the requester's payment is held when a worker is accepted and released only when the job is marked complete, reducing non-payment risk for workers.
- Add a formal dispute-resolution process (as reflected in the activity diagram) so the Administrator can adjudicate disagreements about job completion or quality before funds are released.
- Require ID verification in addition to certificate verification for workers, and consider optional background checks for categories involving home access or vulnerable people.

Matching & Discovery

- Add structured job categories and tags (already partially reflected in the Category entity) so workers can filter by trade/skill rather than only free-text search.
- Support radius-based or map-based job search using geolocation, since urgency and proximity are central to the platform's value proposition.
- Add a recommendation/matching engine that proactively surfaces jobs to online, nearby, skill-matched workers instead of relying purely on browsing.

Trust Signals & Retention

- Track an aggregate rating and a badge (e.g. "Top Rated Worker") rather than only per-job ratings, to help requesters make faster hiring decisions.
- Allow requesters to save/favorite workers they've hired before, to support repeat engagements.
- Define a cancellation and no-show policy with clear consequences for both roles.

Notifications & Communication

- Specify push notifications (e.g. Firebase Cloud Messaging) in addition to in-app notifications for time-sensitive events such as new applicants or job offers.
- Add read receipts and typing indicators to the chat feature to make coordination clearer for time-sensitive, in-person work.

Administration & Analytics

- Expand the Administrator's analytics use case to include job completion rate, average time-to-fill, revenue, and category demand — useful operational metrics not currently itemized.
- Add a support-ticket system so users can escalate issues directly to the Administrator, rather than the Administrator only reacting to reports.

Security & Non-Functional Requirements

- Add explicit non-functional requirements: role-based access control, token-based session management, encryption of stored CVs/certificates, and rate limiting on public endpoints.
- Add password reset / multi-factor authentication as an explicit use case, since it is a common gap in early-stage SRS documents.
- Consider GDPR/POPIA-style data-privacy handling for uploaded personal documents (CVs, certificates, ID verification), including retention and deletion policies.

8. Summary

The five diagrams in this document — use case, activity, class, ERD, and system architecture — cover the design artifacts listed for Phase 2 of the WorkLink development plan, aside from UI wireframes. Together with the improvement suggestions in Section 7, they give a implementation-ready starting point for Phase 3 (Firebase setup, authentication, and Java EE backend development).